Node.js — Day 9

Synchronous vs Asynchronous File Operations

Learning goal: Understand the difference between asynchronous **fs.readFile()** and synchronous **fs.readFileSync()**, and why blocking I/O matters in a Node.js backend.

> **Connection:** This builds on asynchronous programming, the Event Loop, blocking/non-blocking behavior, and the fs module.

1. fs.readFile() — Asynchronous

fs.readFile() reads a file asynchronously. Node.js starts the file operation and continues executing other code instead of waiting for the file result.

```
const fs = require("fs");

console.log("Start");

fs.readFile("./message.txt", "utf8", (err, data) => {
  console.log(data);
});

console.log("End");
```

If the file contains **Hello!**, the typical output order is:

```
Start
End
Hello!
```

The callback runs after the file read is ready.

2. fs.readFileSync() — Synchronous

fs.readFileSync() reads a file synchronously. Node.js waits for the operation to finish before moving to the next line.

```
const fs = require("fs");

console.log("Start");

const data = fs.readFileSync("./message.txt", "utf8");

console.log(data);

console.log("End");
```

If the file contains **Hello!**, the output is:

```
Start
Hello!
End
```

3. The Key Difference

readFile()	readFileSync()
Asynchronous	Synchronous

readFile()	readFileSync()
Continues without waiting for the result	Waits until the operation finishes
Result handled through a callback	Returns the file contents directly
Generally preferred for server-side I/O	Can block the Node.js process

4. Why Synchronous I/O Can Be a Problem

If a synchronous operation takes a long time, it can block the Node.js process from continuing with other work.

```
Request 1
   ↓
synchronous file operation
   ↓
WAIT...
   ↓
Other work has to wait
```

Therefore, asynchronous I/O is generally preferred for file operations in a Node.js server when there is no reason to block.

5. Simple Mental Model

```
readFile()
   ↓
ASYNC
   ↓
Start → continue → callback later

readFileSync()
   ↓
SYNC
   ↓
Wait → finish → continue
```

6. Important Rules

• **readFile()** is asynchronous.

• **readFileSync()** is synchronous.

• Asynchronous reads handle results through a callback.

• Synchronous reads return the result directly.

• Synchronous I/O can block the Node.js process.

• For server-side file I/O, asynchronous operations are generally preferred when you do not need to block execution.

• The difference connects directly to the Event Loop and blocking/non-blocking concepts.

7. Read the Exact Code

Always determine output from the actual code. For example:

```
console.log("A");

const data = fs.readFileSync("./message.txt", "utf8");

console.log("B");
```

This prints only **A** and **B** because **data is not logged**. The file is read and stored in the variable, but it is not displayed.

> If the code contains **console.log(data)**, the file contents will appear between A and B.

8. Practical Backend Context

Most structured application data is commonly obtained from databases or external APIs. The **fs** module is for filesystem I/O: local files, generated reports, CSVs, PDFs, uploads, logs, and other file-related tasks.

Production applications may also use cloud/object storage, so fs is one part of Node.js I/O rather than the main way application data is retrieved.

9. Day 9 Quick Recap

fs.readFile() → asynchronous → callback → generally preferred for server-side file I/O.

fs.readFileSync() → synchronous → waits → can block.

Sync = wait.

Async = start the operation and handle the result later.

10. Practice Completed

Day 9: **2 / 2 questions correct** after correcting one question where the code did not actually print the file data. ■

Important learning point: reason from the exact code, not an assumed output.

Key Takeaway

> **For backend development, understanding sync vs async I/O is more important than memorizing fs methods. readFile() is asynchronous and generally suitable for server I/O; readFileSync() blocks until the operation finishes and should be used carefully in server code.**